



UAVLogBook

Synology DS412+ Docker Install Guide

Purpose-built for DSM 6.2 · Docker 20.10 · Kernel 3.10 · x86_64

Version 1.2 · June 2025

Generic Docker setup	This Synology setup
mysql:8.0	mariadb:10.6 (kernel 3.10 compatible)
php:8.2-apache	php:7.4-apache (DSM 6.2 glibc compatible)
docker compose (v2, space)	docker-compose (v1, hyphen)
Compose version: '3'	Compose version: '2'
Node builder container	Build on workstation, copy dist/ to NAS
Nginx reverse proxy	Apache handles everything directly
service_completed_successfully	depends_on only (v1 limitation)

Table of Contents

1	Prerequisites & RAM Check	3
2	Install Docker on DSM 6.2	4
3	Build the Web App (on your computer)	5
4	Prepare the NAS Directory Structure	6
5	Configure .env	7
6	Run the Install Script	8
7	Manual Setup (without install script)	9
8	First Login	12
9	Day-to-Day Commands	13
10	Updating	14
11	Backup & Restore	15
12	Troubleshooting	16

1 Prerequisites & RAM Check

■ IMPORTANT

The DS412+ ships with only 1 GB RAM. UAVLogBook needs minimum 2 GB. MariaDB alone requires ~300 MB, PHP/Apache ~256 MB, and the OS itself uses ~400 MB. You MUST upgrade to 4 GB RAM before proceeding. This is a simple, cheap hardware change.

RAM Upgrade (required if you have 1 GB)

The Atom D2700 supports up to 4 GB. Any DDR3 SO-DIMM 1333 MHz will work.

RAM Check

Recommended part: Kingston KCP313SS8/4 — (4GB DDR3 1333MHz SO-DIMM)

Cost: approximately €20 / \$22 USD

Guide: <https://jussiroyne.com/2019/08/upgrading-synology-ds412-ram-to-4-gb/>

```
# After installing RAM, verify in DSM:
# Control Panel → Info Center → General → Physical Memory
# Should show: 4.00 GB
```

```
# Or via SSH:
```

```
$ free -m
```

	total	used	free	
Mem:	3855	xyz	xyz	<- Should show ~3855 MB

What you need before starting

✓	Synology DS412+ with DSM 6.2	Required — this guide is for DS412+ only
✓	4 GB RAM installed	See upgrade guide above
✓	Docker package installed in DSM	Install from Package Center
✓	SSH access enabled	Control Panel → Terminal & SNMP → Terminal
✓	SSH client	PuTTY (Windows) or Terminal (macOS/Linux)
✓	Node.js on your workstation	nodejs.org — to build the React app
✓	SCP or file transfer tool	WinSCP (Windows) or scp command (macOS/Linux)
■	Anthropic API key	Optional — enables AI log format detection

2 Install Docker on DSM 6.2

Install Docker from Package Center

- Open DSM web interface in your browser (<http://your-nas-ip:5000>)
- Go to Package Center
- Search for 'Docker'
- Click Install
- Wait for installation to complete — Docker status should show 'Running'

Docker version on DSM 6.2

Synology packages Docker 20.10.3 for DSM 6.2. This is the last version supported on DSM 6.2 and cannot be upgraded further via Package Center. This version is sufficient to run UAVLogBook.

Enable SSH access

- DSM → Control Panel → Terminal & SNMP → Terminal tab
- Check 'Enable SSH service'
- Port: 22 (default)
- Click Apply

Connect via SSH

SSH session

```
# Windows (PuTTY): enter your NAS IP, port 22, login as admin
```

```
# macOS / Linux:
```

```
$ ssh admin@192.168.1.50
```

```
admin@192.168.1.50's password: [your DSM admin password]
```

```
# Verify Docker is working:
```

```
$ docker --version
```

```
Docker version 20.10.3, build ...
```

```
$ docker-compose --version
```

```
docker-compose version 1.28.x, build ...
```

```
# If docker-compose is not found, find it:
```

```
$ ls /var/packages/Docker/target/usr/bin/docker-compose
```

```
$ export PATH=$PATH:/var/packages/Docker/target/usr/bin
```

docker-compose location on Synology

On DSM 6.2, docker-compose is installed at `/var/packages/Docker/target/usr/bin/docker-compose` — not in the standard PATH. If the command is not found, add it to PATH as shown above, or use the full path.

3 Build the Web App (on your workstation)

The DS412+ cannot run Node.js — too little RAM for the build process. Build the React web app on your computer, then copy the output to the NAS.

■ IMPORTANT

Do this step on your COMPUTER (Windows/macOS/Linux), NOT on the Synology. Node.js must be installed on your computer first (nodejs.org).

1

Extract the UAVLogBook zip on your computer

```
# Navigate to the extracted folder
$ cd uavlogbook_v3/web
```

2

Create and edit the environment file

Set VITE_API_URL to your NAS IP address and port (default 8080).

```
$ cp .env.example .env.local

# Edit .env.local – change to your NAS IP:
VITE_API_URL=http://192.168.1.50:8080/api/v1
#               ^^^^^^^^^^^^^^^^^^^^^^ <- your NAS IP
```

3

Install dependencies and build

This runs on your computer — takes 2–5 minutes.

```
$ npm install
$ npm run build

# On success you will see:
# dist/index.html
# dist/assets/index-xxxx.js
# dist/assets/index-xxxx.css
```

4

Copy the built files to the NAS

Copy everything inside web/dist/ to the NAS.

```
# macOS / Linux:
$ scp -r dist/* admin@192.168.1.50:/volume1/docker/uavlogbook/www/

# Windows (PowerShell + OpenSSH):
> scp -r dist\* admin@192.168.1.50:/volume1/docker/uavlogbook/www/

# Alternative: use WinSCP (Windows) or Cyberduck (macOS)
# Upload the CONTENTS of dist/ to: /volume1/docker/uavlogbook/www/
# (Upload what's INSIDE dist/, not the dist/ folder itself)
```

Verify the copy worked

After copying, SSH into the Synology and check: `ls /volume1/docker/uavlogbook/www/` — you should see `index.html` and an `assets/` directory. If you only see the `dist/` directory itself, you copied the folder instead of its contents.

4 Prepare the NAS Directory Structure

All UAVLogBook data lives under `/volume1/docker/uavlogbook/`. This keeps it separate from other Docker apps and easy to back up.

Create directories via SSH

```
SSH on Synology
# SSH into your Synology first
$ ssh admin@192.168.1.50

# Create the full directory structure
$ mkdir -p /volume1/docker/uavlogbook/{db,init,server,www,uploads/logs,uploads/videos}

# Verify
$ ls /volume1/docker/uavlogbook/
db  init  server  uploads  www

$ ls /volume1/docker/uavlogbook/uploads/
logs  videos
```

Directory purpose

<code>/volume1/docker/uavlogbook/</code>	Project root — all docker files go here
<code>db/</code>	MariaDB data files — do not touch
<code>init/</code>	schema.sql — auto-imported on first start
<code>server/</code>	PHP source files (copied from server/)
<code>www/</code>	React web app (copied from web/dist/)
<code>uploads/logs/</code>	Flight log files uploaded by users
<code>uploads/videos/</code>	FPV video files uploaded by users
<code>docker-compose.yml</code>	Container definitions
<code>Dockerfile.synology</code>	PHP 7.4 image build instructions
<code>apache-synology.conf</code>	Apache virtual host configuration
<code>.env</code>	Passwords and secrets (you create this)

Copy project files to NAS

From your computer

From your computer – copy the server files:

```
$ scp -r server/* admin@192.168.1.50:/volume1/docker/uavlogbook/server/
```

Copy the Docker files from docker/synology/:

```
$ scp docker/synology/docker-compose.yml      admin@192.168.1.50:/volume1/docker/uavlogbook/
```

```
$ scp docker/synology/Dockerfile.synology      admin@192.168.1.50:/volume1/docker/uavlogbook/
```

```
$ scp docker/synology/apache-synology.conf     admin@192.168.1.50:/volume1/docker/uavlogbook/
```

Copy the database schema (for auto-import):

```
$ scp server/db/schema.sql                    admin@192.168.1.50:/volume1/docker/uavlogbook/init/
```

5 Configure .env

Create the .env file on the Synology with your passwords and settings.

Create .env via SSH

```
SSH on Synology
# SSH into Synology
$ ssh admin@192.168.1.50

# Navigate to project directory
$ cd /volume1/docker/uavlogbook

# Generate a strong JWT secret
$ openssl rand -hex 32
a8f3d2c1b4e5f6a7b8c9d0e1f2a3b4c5d6e7f8a9b0c1d2e3f4a5b6c7d8e9f0a1

# Create the .env file (paste your generated JWT secret below)
$ cat > .env << 'EOF'
MYSQL_ROOT_PASSWORD=YourStrongRootPasswordHere
MYSQL_DATABASE=uavlogbook
MYSQL_USER=uavlogbook_user
MYSQL_PASSWORD=YourStrongAppPasswordHere
JWT_SECRET=paste-your-64-char-openssl-output-here
ANTHROPIC_API_KEY=
ALLOWED_ORIGINS=http://192.168.1.50:8080
APP_PORT=8080
EOF

# Secure the file – only owner can read
$ chmod 600 .env

# Verify it looks correct
$ cat .env
```

ALLOWED_ORIGINS

Set ALLOWED_ORIGINS to your NAS's actual IP address (not localhost) so the mobile app and other devices on your network can connect. Use your NAS's fixed local IP, e.g.

http://192.168.1.50:8080. If you have a domain pointing to your NAS, add it comma-separated: http://192.168.1.50:8080,https://yourdomain.com

6 Run the Install Script (Automatic Method)

If you copied the `install_synology.sh` script to the NAS, it automates steps 4–8. It asks you questions, creates directories, copies files, writes `.env`, builds the image, starts containers, and verifies the API.

```
SSH on Synology
# SSH into Synology
$ ssh admin@192.168.1.50

# Copy the install script first (from your computer):
# scp docker/synology/install_synology.sh admin@192.168.1.50:/volume1/docker/

# Make it executable and run it
$ chmod +x /volume1/docker/install_synology.sh
$ bash /volume1/docker/install_synology.sh

# The script will:
#   ✓ Check Docker version and RAM
#   ✓ Ask for your passwords
#   ✓ Create all directories
#   ✓ Generate JWT secret automatically
#   ✓ Build the PHP 7.4 Docker image
#   ✓ Start containers
#   ✓ Wait for MariaDB to be ready
#   ✓ Verify the API is responding

# After the script completes:
# Open: http://192.168.1.50:8080
```

Image build time

Building the PHP 7.4 Docker image takes 10–20 minutes on the Atom D2700. This only happens once. Subsequent `docker-compose up` commands start in seconds. Do not interrupt the build — watch progress with the logs that appear in the terminal.

7 Manual Setup (step by step without script)

Use this if you prefer to do each step by hand, or if the install script failed partway through.

1

Verify all files are in place

```
$ ls /volume1/docker/uavlogbook/
Dockerfile.synology  apache-synology.conf  docker-compose.yml
.env  db/  init/  server/  www/  uploads/

$ ls /volume1/docker/uavlogbook/init/
schema.sql

$ ls /volume1/docker/uavlogbook/www/
index.html  assets/
```

2

Navigate to the project directory

```
$ cd /volume1/docker/uavlogbook
```

3

Build the Docker image

This downloads php:7.4-apache and installs extensions. Takes 10–20 minutes.

```
$ docker-compose build app

# Watch for: Successfully built xxxxxxxx
# If it fails with an error, check internet connectivity:
$ curl -I https://hub.docker.com
```

4

Start containers

```
$ docker-compose up -d

# Verify both containers started:
$ docker-compose ps
```

Name	Command	State	Ports
uavlogbook_db	mysqld ...	Up	3306/tcp
uavlogbook_app	apache2-foreground	Up	0.0.0.0:8080->80/tcp

5

Wait for MariaDB to initialise

On first start, MariaDB creates the database and imports schema.sql. This takes 60–120 seconds. Watch the logs:

```
$ docker-compose logs -f db

# Wait until you see:
# [Note] mysqld: ready for connections.

# Press Ctrl+C to stop watching logs
```

6

Verify the API is working

```
$ curl -s -X POST http://localhost:8080/api/v1/auth/login \
-H 'Content-Type: application/json' \
-d '{"email":"test","password":"test"}'

# Expected response:
# {"error":"Invalid credentials"}

# This confirms: Apache is running, PHP is working, MariaDB is connected
```

8 First Login

Open the web app

```
# Open in your browser (replace with your NAS IP):  
http://192.168.1.50:8080  
  
# You should see the UAVLogBook login screen.  
# If you see a blank page, check Section 12 (Troubleshooting).
```

Register your account

- Click Register
- Enter your email address
- Choose a strong password (minimum 8 characters)
- Enter a display name (e.g. 'Pilot')
- Click Create Account
- You are now logged in — the first account created becomes admin

Mobile app connection

To use the mobile app (iOS/Android), edit mobile/src/api.js on your computer:

```
mobile/src/api.js  
# In mobile/src/api.js, change line 6:  
export const API_BASE_URL = 'http://192.168.1.50:8080/api/v1';  
#                               ^^^^^^^^^^^^^^^^^^^^^ <- your NAS IP  
  
# Then build the app with Expo:  
$ cd mobile && npm install && npx expo start  
# Scan the QR code with Expo Go app on your phone
```

9 Day-to-Day Commands

All commands run via SSH from /volume1/docker/uavlogbook/

Container management

<code>docker-compose ps</code>	Show running containers and their status
<code>docker-compose up -d</code>	Start all containers in background
<code>docker-compose down</code>	Stop and remove containers (data kept)
<code>docker-compose restart app</code>	Restart only the PHP/Apache container
<code>docker-compose restart db</code>	Restart only the MariaDB container

Logs

<code>docker-compose logs -f</code>	Tail all container logs
<code>docker-compose logs -f app</code>	Tail PHP/Apache logs only
<code>docker-compose logs -f db</code>	Tail MariaDB logs only
<code>docker-compose logs --tail=50 app</code>	Last 50 lines of app logs

Troubleshooting

<code>docker exec -it uavlogbook_app bash</code>	Shell inside PHP container
<code>docker exec -it uavlogbook_db bash</code>	Shell inside MariaDB container
<code>docker stats --no-stream</code>	Show CPU/RAM usage of containers

10 Updating

Update PHP server code

```
From your computer, then SSH
# Copy new server files from your computer to NAS:
$ scp -r server/* admin@192.168.1.50:/volume1/docker/uavlogbook/server/

# Restart the app container to load the new files:
$ ssh admin@192.168.1.50
$ cd /volume1/docker/uavlogbook
$ docker-compose restart app

# Verify it's working:
$ docker-compose logs -f app
```

Update web app (after changing web/src/)

```
From your computer
# On your computer – rebuild the React app:
$ cd web && npm run build

# Copy new files to NAS:
$ scp -r dist/* admin@192.168.1.50:/volume1/docker/uavlogbook/www/

# No container restart needed – Apache serves static files directly
# Hard-refresh your browser: Ctrl+Shift+R
```

Rebuild Docker image (after Dockerfile changes)

```
SSH on Synology
$ ssh admin@192.168.1.50
$ cd /volume1/docker/uavlogbook
$ docker-compose down
$ docker-compose build --no-cache app
$ docker-compose up -d
```


11 Backup & Restore

Database backup

```
SSH on Synology
$ ssh admin@192.168.1.50
$ cd /volume1/docker/uavlogbook

# Create a backup (reads passwords from .env automatically)
$ source .env
$ docker exec uavlogbook_db mysqldump \
    -u$MYSQL_USER -p$MYSQL_PASSWORD $MYSQL_DATABASE \
    > /volume1/docker/uavlogbook/backup_$(date +%Y%m%d).sql

# Verify the backup
$ ls -lh /volume1/docker/uavlogbook/backup_*.sql
```

Database restore

```
SSH on Synology
$ source .env
$ docker exec -i uavlogbook_db mysql \
    -u$MYSQL_USER -p$MYSQL_PASSWORD $MYSQL_DATABASE \
    < /volume1/docker/uavlogbook/backup_20250601.sql
```

Backup uploads and videos

Flight logs and videos are in /volume1/docker/uavlogbook/uploads/. Back them up using DSM Hyper Backup or manually:

```
SSH on Synology
# In DSM: Hyper Backup → + → Data backup task
# Source folder: /volume1/docker/uavlogbook/
# Destination: external drive, another NAS, or cloud

# Or manually copy to an external USB drive:
$ cp -r /volume1/docker/uavlogbook/uploads/ /volumeUSB1/usbshare/uavlogbook_backup/
```

Automated daily backup (via DSM Task Scheduler)

DSM → Control Panel → Task Scheduler → Create → Scheduled task → User-defined script:

DSM Task Scheduler script

```
#!/bin/bash

cd /volume1/docker/uavlogbook

source .env

docker exec uavlogbook_db mysqldump \
    -u$MYSQL_USER -p$MYSQL_PASSWORD $MYSQL_DATABASE \
    > /volume1/docker/uavlogbook/backups/db_$(date +%Y%m%d_%H%M%S).sql

# Keep last 30 backups

ls -t /volume1/docker/uavlogbook/backups/db_*.sql | tail -n +31 | xargs rm -f
```

12 Troubleshooting

docker-compose: command not found

The docker-compose binary is not in PATH on Synology.

```
$ export PATH=$PATH:/var/packages/Docker/target/usr/bin
$ docker-compose --version

# Make permanent – add to /root/.bashrc:
$ echo 'export PATH=$PATH:/var/packages/Docker/target/usr/bin' >> /root/.bashrc
```

OOM (Out of Memory) — containers keep crashing

Not enough RAM. Upgrade the DS412+ RAM to 4 GB.

```
# Check current RAM:
$ free -m

# Check which container is being killed:
$ dmesg | grep -i 'oom\|killed'
$ docker-compose logs db | grep -i 'error\|killed'
```

Blank white page in browser

Web app files not copied correctly to /volume1/docker/uavlogbook/www/

```
# Check what's in www/:
$ ls /volume1/docker/uavlogbook/www/
# Should show: index.html assets/

# If it shows: dist/ <- wrong! You copied the folder not contents
# Fix:
$ mv /volume1/docker/uavlogbook/www/dist/* /volume1/docker/uavlogbook/www/
$ rmdir /volume1/docker/uavlogbook/www/dist
```

API returns 500 or connection refused

PHP cannot connect to MariaDB. Check if MariaDB is ready.

```
$ docker-compose logs db | tail -20
# Look for: ready for connections

# Check PHP error log:
$ docker exec uavlogbook_app cat /var/log/apache2/uavlogbook_error.log

# Common cause: .env MYSQL_PASSWORD does not match
# Fix: docker-compose down, fix .env, docker-compose up -d
```

MariaDB fails to start — init error

The db/ directory has leftover files from a previous install. Clear it.

```
$ docker-compose down
$ rm -rf /volume1/docker/uavlogbook/db/*
# WARNING: this deletes ALL database data!
$ docker-compose up -d
# MariaDB will re-import schema.sql from init/
```

Build fails: no space left on device

The system volume is full. Docker images need ~1.5 GB of space.

```
# Check space:
$ df -h /
$ docker system df

# Clean up old Docker images:
$ docker image prune -f
$ docker system prune -f
```

Mobile app cannot connect to NAS

CORS or network issue. The API_BASE_URL in mobile/src/api.js must match your NAS IP.

```
# Verify from your phone's browser:
http://192.168.1.50:8080/api/v1/auth/login

# Check ALLOWED_ORIGINS in .env:
ALLOWED_ORIGINS=http://192.168.1.50:8080

# After editing .env:
$ docker-compose restart app
```